Programming Fundamentals

Week 8 — Monday: Errors and Exception Handling

Programming Fundamentals Team

SETU

2026-11-05

Today: Errors Are Part of Programming

By the end of today you should be able to:

- recognise syntax, runtime and logic errors
- read a Python traceback
- handle specific exceptions using `try` and `except`
- distinguish validation from exception handling
- make our JSON-based programs more resilient

Warm-up — What Could Go Wrong?

```
age = int(input("Age: "))
if age >= 18:
    print("Adult")
```

Predict the result for `21`, `-2`, and `hello`.

Warm-up — What Could Go Wrong?

```
age = int(input("Age: "))
if age >= 18:
    print("Adult")
```

Predict the result for `21`, `-2`, and `hello`.

Not every problem is an exception: some inputs are valid numbers but unsuitable.

Entering an int

What would happen if a user entered a product name where a product code (integer) was expected.

```
product_code = int(input("Product code: "))
print("You entered:", product_code)
```

Try: enter `123`, then `iPhone11`.

Entering an int

What would happen if a user entered a product name where a product code (integer) was expected.

```
product_code = int(input("Product code: "))
print("You entered:", product_code)
```

Try: enter `123`, then `iPhone11`.

`int("iPhone11")` raises a `ValueError`.

Three Kinds of Problems

- **Syntax error:** Python cannot parse the program.
- **Runtime error:** the program starts but an operation fails.
- **Logic error:** the program runs but gives the wrong answer.

Question: Which one might go unnoticed if we only check whether the program runs?

Three types of Errors

Syntax error	Runtime error	Logic error
Code cannot start	Code starts, then fails	Code runs, wrong answer
missing :	int("abc")	wrong condition

Question: Which type can pass unnoticed even if the program runs successfully?

Spot the Syntax Error

```
mark = 55
if mark >= 40
    print("Pass")
```

Find the missing character before running.

Spot the Syntax Error

```
mark = 55
if mark >= 40
    print("Pass")
```

Find the missing character before running.

A colon (`:`) is missing after the condition.

A Runtime Error

```
count = int(input("How many dogs? "))
print("You entered", count)
```

Enter `three`.

What does Python tell you? Where should you start reading?

A Logic Error

```
mark = 75
if mark < 40:
    print("Pass")
else:
    print("Fail")
```

This program runs. Why is its output wrong?

Fix the condition, then test `39`, `40`, and `75`.

What Is an Exception?

An **exception** is an object that signals a problem while Python is executing.

- Normal execution is interrupted.
- Python searches for a matching exception handler.
- If no handler is found, the program stops and displays a traceback.
- If handled appropriately, the program may continue sensibly.

Handling an exception is different from fixing the cause. The programmer must decide how to recover.

How to Read a Traceback

```
def average(total, count):
    return total / count

print(average(100, 0))
```

Look for:

1. The **exception type** at the bottom
2. The **file, line and function** where it happened
3. The operation that caused it

Here: `ZeroDivisionError` — dividing by zero.

Java Exceptions → Python Exceptions

Python	Typical cause	`ValueError`
Converting `"abc"` to `int`	`ZeroDivisionError`	Dividing by zero
`IndexError`	Invalid list index	`AttributeError`
Calling a method on `None`		

Common Exceptions — Part 1

- `ValueError`: conversion fails, e.g. `int("abc")`
- `TypeError`: incompatible types, e.g. `"Age: " + 4`
- `NameError`: a name has not been defined
- `IndexError`: list index is out of range

Common Exceptions — Part 2

- `KeyError`: dictionary key does not exist
- `FileNotFoundError`: a file opened for reading is missing
- `ZeroDivisionError`: division by zero
- `json.JSONDecodeError`: invalid JSON content

Link to Week 7: Which errors might occur when loading `dogs.json`?

Predict Before Running

```
dogs = [{"name": "Buddy"}]
print(dogs[1]["age"])
```

Which exception happens **first**?

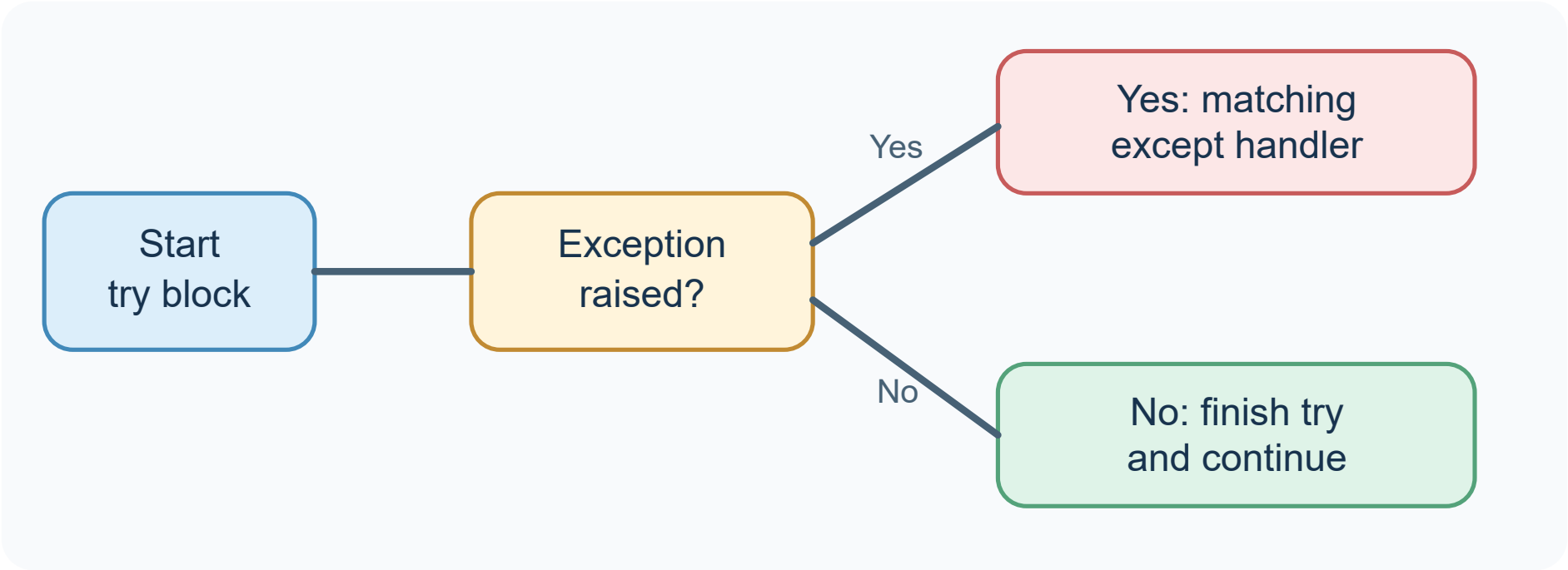
Predict Before Running

```
dogs = [{"name": "Buddy"}]
print(dogs[1]["age"])
```

Which exception happens **first**?

`IndexError` — Python cannot access `dogs[1]`, so it never checks the key.

Exception Flow — What Happens Next?



Question: What happens to the remaining statements inside `try` after an exception is raised?

Pythons except

```
try:
    product_code = int(input("Product code: "))
except ValueError as error:
    print("Invalid product code:", error)
```

Python uses indentation, `except`, and optionally `as error` to access the exception object.

Handling an Exception

```
try:
    age = int(input("Age: "))
    print("Age:", age)
except ValueError:
    print("Please enter a whole number.")
```

`try` contains an operation that might fail.

`except ValueError` handles that **specific** failure.

What Does `try` Actually Do?

1. Python starts executing the `try` block.
2. If an exception occurs, remaining lines in that block are skipped.
3. A matching `except` block runs.
4. Execution continues after the whole statement.

Exception handling does not automatically correct invalid data. We must decide what the program should do next.

Validation Versus Exceptions

```
age = int(input("Dog age: "))
if 0 <= age <= 25:
    print("Accepted")
else:
    print("Age must be 0 to 25")
```

`-3` is a valid integer but an **invalid age**.

`abc` cannot be converted to an integer: **exception**.

Combine a Loop with `try`

```
while True:
    try:
        age = int(input("Dog age (0–25): "))
        if 0 <= age <= 25:
            break
        print("Age must be 0 to 25")
    except ValueError:
        print("Enter a whole number")
print("Accepted:", age)
```

Test: abc, -1, 26, 4.

Live Coding — Turn It into a Function

```
def get_valid_age():
    while True:
        try:
            age = int(input("Age (0–25): "))
            if 0 <= age <= 25:
                return age
            print("Out of range")
        except ValueError:
            print("Whole numbers only")
```

Why does `return` remove the need for `break`?

Be Specific About What You Catch

Prefer:

```
except ValueError:
    print("Please enter a number")
```

Avoid catching every exception with a bare `except:`.

Why? It can hide programming mistakes that we need to fix.

`else`: Only When No Exception Occurs

```
try:
    number = int(input("Number: "))
except ValueError:
    print("Not a whole number")
else:
    print("Double:", number * 2)
```

Predict what happens for `5` and `five`.

`finally`: Runs Either Way

```
try:
    number = int(input("Number: "))
except ValueError:
    print("Invalid number")
finally:
    print("Input attempt finished")
```

Useful for cleanup; `with open()` already manages closing files for us.

Python JSON

```python
import json

products = [{"code": 101, "name": "Mouse"}]
with open("products.json", "w", encoding="utf-8") as file:
    json.dump(products, file, indent=4)
```

Possible failures: permission denied, folder missing, disk problems.

The `with` statement closes the file automatically, including when an exception occurs.

Handling Save Errors in the Menu

```
import json

def save_products(products):
    try:
        with open("products.json", "w", encoding="utf-8") as file:
            json.dump(products, file, indent=4)
    except OSError as error:
        print("Could not save products:", error)
        return False
    return True
```

The menu can check the returned `True` or `False` before displaying a success message.

Question: Why should we avoid printing “Saved!” before we know the operation succeeded?

Back to Persistence: Missing Files

```
import json

def load_dogs():
    try:
        with open("dogs.json", "r", encoding="utf-8") as file:
            return json.load(file)
    except FileNotFoundError:
        print("No saved data yet")
        return []
```

Question: Why is an empty list a sensible starting point when the file has never been created?

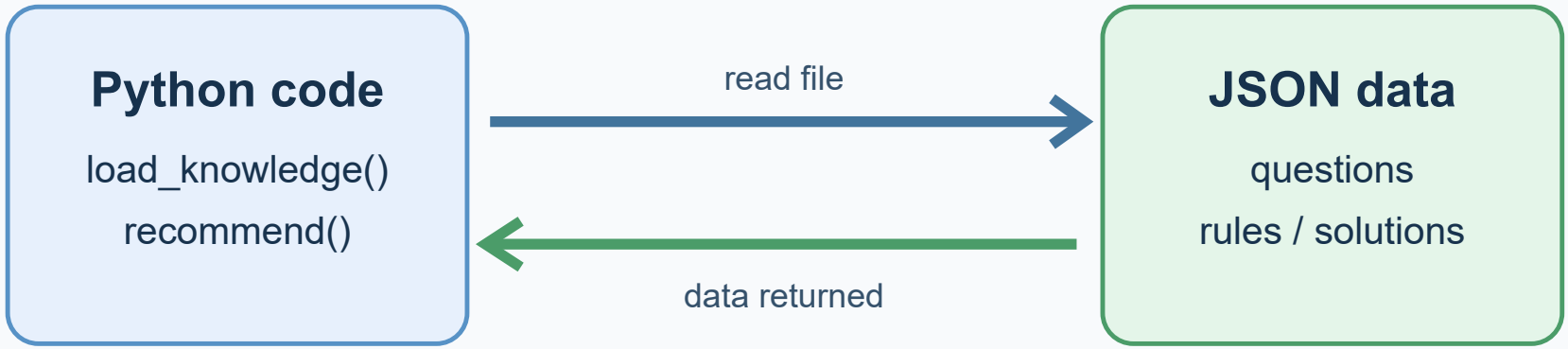
What If the JSON Is Damaged?

```
except json.JSONDecodeError:
    print("Saved data is not valid JSON")
    return None
```

This branch belongs in `load_dogs()` after the `FileNotFoundError` branch.

Do not automatically overwrite a damaged JSON file with an empty list. Use `None` to signal that loading failed and stop saving until the issue is resolved.

Our PC Troubleshooter Knowledge Base



The JSON file contains our knowledge. Python contains the logic that reads and uses it.

Our PC Troubleshooter Knowledge Base

If the file is missing, invalid, or inaccessible, our program needs to respond appropriately rather than crash unexpectedly.

Loading JSON — Handle Different Failures

```
import json

def load_knowledge(filename):
    try:
        with open(filename, "r", encoding="utf-8") as file:
            return json.load(file)
    except FileNotFoundError:
        print("Knowledge file not found")
    except json.JSONDecodeError:
        print("Knowledge file contains invalid JSON")
    except OSError as error:
        print("Cannot read knowledge file:", error)
    return None
```

Important: `None` signals failure. Don't overwrite a broken knowledge base with empty data.

Mini Activity — Protect a Calculation

```
def average(total, count):
    return total / count

print(average(20, 0))
```

Modify the caller so it catches `ZeroDivisionError`.

Then discuss: should we **also validate** that `count` is greater than zero?

Test Cases — Not Just Happy Paths

For `get_valid_age()`, test:

- **Normal:** `4`
- **Boundary:** `0`, `25`
- **Out of range:** `-1`, `26`
- **Wrong type:** `hello`, blank input

Record the expected behaviour before running each test.

Quick Check — Can We Recover?

Match the problem to the best response:

1. User types `cat` for a numeric product code.
2. `knowledge.json` is missing.
3. JSON contains a missing comma.
4. A program recommends the wrong solution but does not crash.

Discuss: Which are exceptions? Which is a logic error? Should the program retry, report the problem, or stop safely?

Monday Recap

- An exception is not the same as a logic error.
- Read the **last line** of a traceback, then follow the relevant file and line references.
- Catch **specific** exceptions.
- Combine exception handling with validation and loops.
- Never silently overwrite damaged saved data.

Wednesday: Find the cause of bugs using the VS Code debugger.